

```
$ cat notes.md
```

DevOps & Cloud [SWE] — Class 11 Notes

LECTURE TITLE Kubernetes Networking & Services

DATE/TIME 8 September 2026, 11:00 AM

DURATION 120 minutes

INSTRUCTOR Nensi Ravaliya

BIG IDEA There are four different ports in a Kubernetes YAML, and each one belongs to a different layer. Get that straight and networking stops being scary.

SOURCES this class's transcript + the YAML in this folder (`pod.yml` , `service.yml` , `backend-rs.yml` , `deployment-v1.yml`) + her **devops-heros** repo

WHAT'S INSIDE — 19 SECTIONS

- 00 ⚡ The 60-Second Version
- 01 📅 Housekeeping
- 02 🗨️ Command Recap — The get / describe / delete Family
- 03 💎 The Four Ports — The Core Of This Class
- 04 💰 Ingress — And Why It's Really About Money
- 05 🔑 Why Service Exists — The Changing-IP Problem
- 06 📝 Writing a Service
- 07 🗨️ Labels & Selectors — The Classic Interview Question
- 08 🧑‍🔬 Lab 1 — Pod + Service + Port Forwarding
- 09 📄 Scaling — And The One Thing You Can't Scale
- 10 📦 ReplicaSet — A pod.yml With Three Extra Fields
- 11 🚀 Deployment & The Four Strategies
- 12 📄 maxSurge & maxUnavailable — The Long Version
- 13 ⚙️ Resources — requests vs limits
- 14 🔥 The OpenShift Outage Story
- 15 👤 DaemonSet — One Pod Per Node
- 16 📄 Homework
- 17 📄 Cheat-Sheet
- 18 📄 Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. theme: Lagoon

00



The 60-Second Version

1. **Four ports, four layers:** `containerPort` (container) → `targetPort` (pod) → `port` (service) → `nodePort` (node/server).
2. **Pod IPs change every restart.** That's why you need a **Service** — a stable front door that finds pods by label.
3. **Labels go on pods. Selectors go on Services, ReplicaSets and Deployments.** The two strings must match exactly.
4. `kubectl describe` is Kubernetes' `docker inspect`.
5. **ReplicaSet = a pod.yaml plus `replicas`, `selector` and `template`.** Strip those three and you're back to a plain pod.
6. **You can scale a ReplicaSet or a Deployment. You cannot scale a Pod.**
7. **Deployment adds what ReplicaSet can't do: migration.** Four strategies — RollingUpdate, Blue-Green, Canary, and one more.
8. `requests` = the minimum you're guaranteed. `limits` = the ceiling. Blow the memory limit and you get **OOMKilled**.
9. **Ingress exists to save money** — one URL routing to thousands of services, instead of one \$200/month load balancer each.



01



Housekeeping



CLASS MOMENT

Her opening: *"This is good — if someone is taking the session before my session, I'm so happy, because my mic is working, everything is working. I don't need to do anything, I just start my session. Even though he took over seven minutes... but that's okay, it's worth it."* 😊

The minikube rule



WATCH OUT

"First task: start minikube. Right now. And don't stop it. So next time when I come to see the output, you can't say to me 'my minikube is just starting.'"

Today's plan, in her words

```
pod lifecycle → selectors & labels → service
→ deployment → daemonset → scaling replicas
→ why you need a ReplicaSet, where it fails, and why that means Deployment
```

SAID IN CLASS

"Anyhow we have to complete this — otherwise you have an extra session." 😊



02 Command Recap — The `get` / `describe` / `delete` Family

She ran through the whole command surface before starting.

The `get` family

```
kubectl get pods
kubectl get namespaces
kubectl get nodes
kubectl get rs           # or: get replicaset
kubectl get deploy       # or: get deployment
kubectl get svc          # or: get service / services
kubectl get all          # ★ everything at once
kubectl get events       # "kind of a log"
kubectl logs <pod>
```

GOOD TO KNOW

"ReplicaSet short form is `rs`. Deployment — `deploy` is fine, `deployment` is also fine. Sometimes `deploy` works, sometimes it doesn't."

😊 "I'll just type `svc` because I don't like to write long long commands."

describe — the **docker inspect** of Kubernetes

LOCK THIS IN

Her mapping, and it's the right way to remember it:

```
DOCKER                                KUBERNETES
docker inspect container <name> →    kubectl describe pod <name>
docker inspect volume <name> →       kubectl describe deploy <name>
                                   →    kubectl describe node <name>
```

 "Same thing we have — not **inspect**, we have a **describe** command."

What **describe pod tells you:** which image, how many resources, how many replicas, which container port — "these information regarding the pod."

delete

```
kubectl delete pod <name>
kubectl delete svc <name>
kubectl delete deploy <name>
kubectl delete rs <name>
```

GOOD TO KNOW

A habit worth copying: "We can also delete by file — **kubectl delete -f <file>** — but 99.99% you are not going to use it, because we never delete the file. We just update the file and do **kubectl apply**."

exec

```
docker exec -it <container> sh → kubectl exec -it <pod> -- sh
```

SAID IN CLASS

"Same command. Just **kubectl**, where we had **docker**. And here we have a pod, not a container."

create vs apply, one more time

	Doesn't exist	Already exists
<code>kubectl create -f</code>	✅ creates	❌ error: already exists
<code>kubectl apply -f</code>	✅ creates	✅ updates

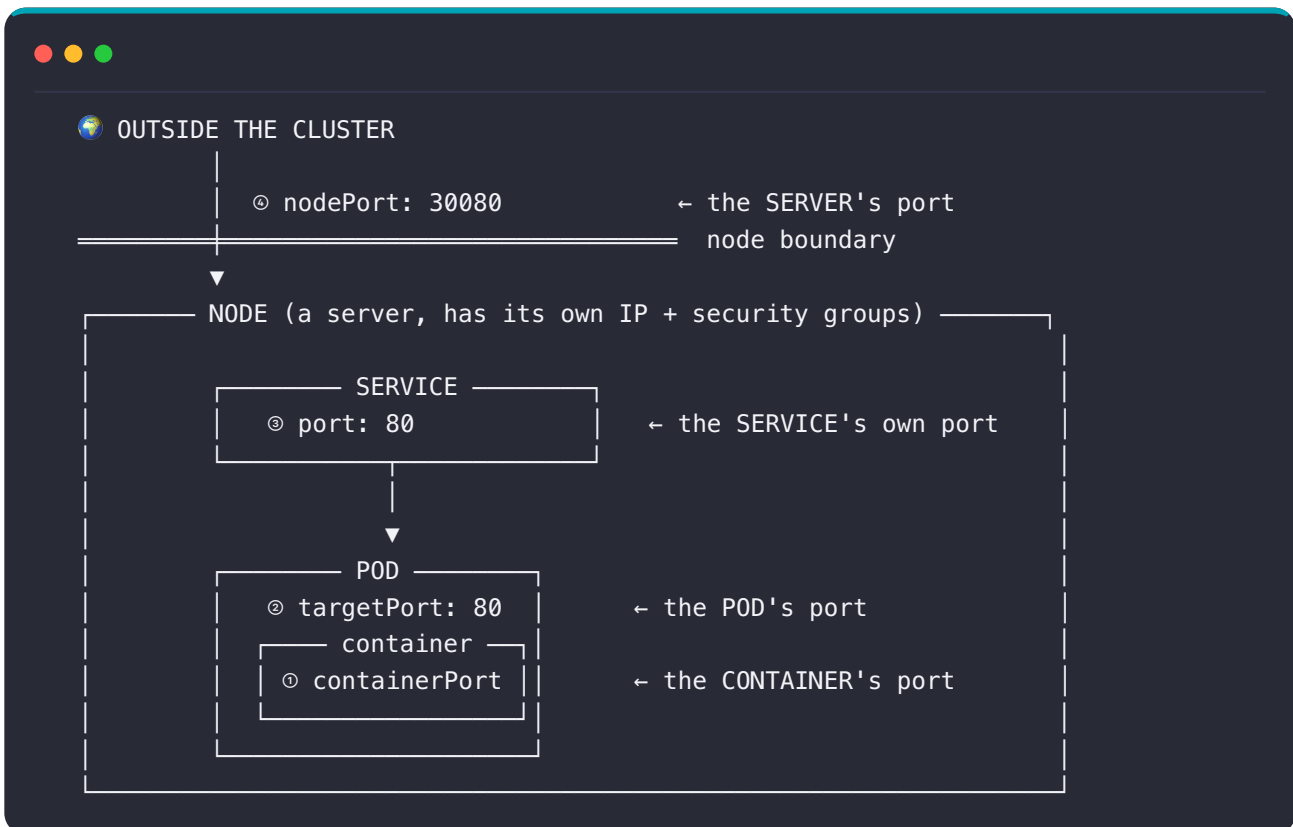


03 💎 The Four Ports — The Core Of This Class

This was the centrepiece. She drew it out because everyone conflates them.

🗣️ SAID IN CLASS

"There are four types of port that you are going to write in your YAML file."



#	Port	Belongs to	Written in
①	<code>containerPort</code>	the container	<code>pod.yaml</code> / a deployment template
②	<code>targetPort</code>	the pod	<code>service.yaml</code>
③	<code>port</code>	the service	<code>service.yaml</code>

#	Port	Belongs to	Written in
④	<code>nodePort</code>	the node (server)	<code>service.yaml</code>

💡 GOOD TO KNOW

Her note on `containerPort` vs `targetPort` : "We are not going to deal with `containerPort` much, because we are not going to create `pod.yaml` usually in big infrastructure. We are going to create `deployment.yaml` , and inside that we mention our `targetPort` ."

✅ So `containerPort` is a learning-stage / small-assessment thing; `targetPort` is what you'll actually use.

🧩 WHY IT WORKS

What a "node" really is, since it underpins `nodePort` :

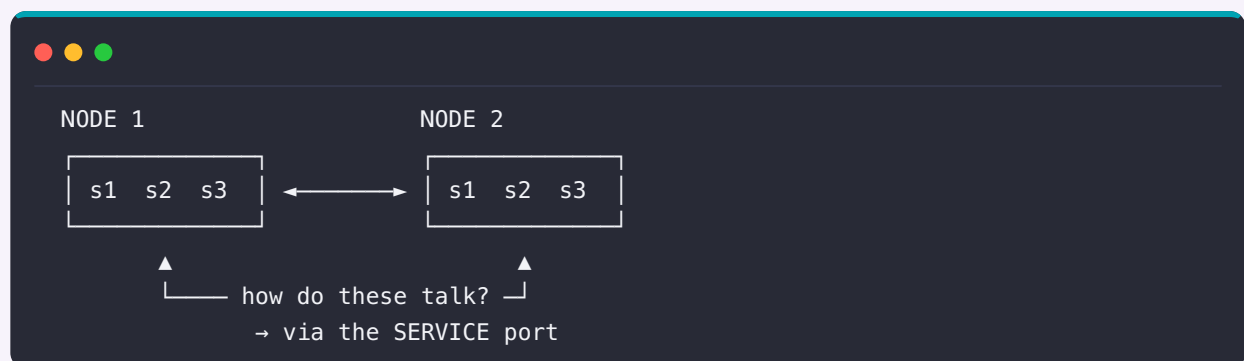
"Node means nothing else — a server. An IP address. You create an EC2 instance, you make it a node, you add it to a cluster. After adding that server inside a cluster, it becomes a node. And a server has its own IP, its own inbound rules, its own security groups where we add our ports."

That's why a `nodePort` opens a real port on a real machine — and why it's in the restricted `30000–32767` range.

Why the `port` (service) one exists at all

🧠 PICTURE THIS

Service-to-service communication. She drew two nodes with three services each:



🗣️ "How are two services communicating with each other? For that we need a service, and for that we need a service port."



04 💰 Ingress — And Why It's Really About Money

She introduced Ingress here, and framed it in a way you won't find in the docs.

🔔 INTERVIEW ALERT

The problem: "When you have thousands of services, each time you are not mentioning the port, not giving the service URL. In AWS, for 1000 services you're creating a thousand load balancers just to route to that particular service."

The arithmetic that makes the case

```
1 load balancer    ≈ $20/month    (her conservative figure)
5 services         = 5 LBs        = $100/month
```

"Actually the services are costing around \$200 per month for ONE service."

```
1,000 services    = 1,000 LBs    = 💰💰💰💰💰
```

🗣️ SAID IN CLASS

"They are running thousands of services. Can you imagine how much bill we are getting just for redirecting one service to a load balancer? So to reduce that cost, they implemented Ingress."

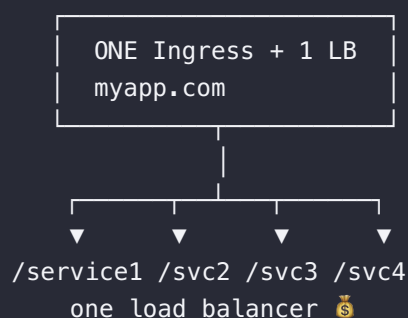
What Ingress actually is

WITHOUT INGRESS

```
LB → service1    💰
LB → service2    💰
LB → service3    💰
LB → service4    💰
...
LB → service1000 💰

1000 load balancers
```

WITH INGRESS



✅ LOCK THIS IN

"Ingress is nothing but a DNS URL. They created one universal URL, and from here — my service name is service1, so they give the service1 URL, service2 URL — like this, thousands of services are working, and they're saving a pretty much amount."

💡 GOOD TO KNOW

The two routing modes she named: **path-based routing** (`myapp.com/payments` → payment service) and **default routing**.

📌 NOTE

Note: this is the **Ingress** that Class 12's folder title promised. It was introduced here, conceptually. The hands-on hasn't happened yet.



05 🔑 Why Service Exists — The Changing-IP Problem

🔔 INTERVIEW ALERT

The whole argument in one picture:

```
YOU CREATE 3 PODS FROM pod.yaml
  p1 → 10.0.0.1      p2 → 10.0.0.2      p3 → 10.0.0.3

p2 crashes (CrashLoopBackOff / Completed / Pending)
  💀

p2 comes back → 10.0.0.47      ← A COMPLETELY DIFFERENT IP
```

🗣️ "Every time when this service goes down, its IP address will change. So how will you know which IP address belongs to my pod? Every time you can't search the IP address — you have a lot of servers."

⚠️ WATCH OUT

And the "do you really do this by hand?" question: "Do you actually run `kubectl` to scale up this pod, scale down that pod, if you have thousands of pods running? Really? Is that a good enough approach? No, right."

✓ LOCK THIS IN

So the hierarchy is built from the bottom up, each layer solving the layer below's problem:



```
pod.yaml      → one pod. IP changes. Unmanageable at scale.
  ↓ "I need many, and I need them replaced automatically"
ReplicaSet    → N pods, always
  ↓ "I need a stable address in front of them"
Service       → one name, finds pods by LABEL, not by IP
  ↓ "I need to change versions without downtime"
Deployment    → manages ReplicaSets + migration strategies
```

🗣️ *"Deployment is managing your services plus ReplicaSet. And Service is actually managing all the replicas."*



06 🍴 Writing a Service



```
apiVersion: v1
kind: Service

metadata:
  name: nginx-service      # convention: <name>-service

spec:
  type: NodePort           # ★ MANDATORY
  selector:
    app: nginx             # ★ must match the pod's LABEL
  ports:
    - port: 80             # ★ MANDATORY – the service's port
      targetPort: 80       # the pod's port
      nodePort: 30080      # only for type: NodePort
```

✓ LOCK THIS IN

Only two fields are truly mandatory: *"Service type is mandatory, and service port is mandatory. Two things."*

🧠 Her method for writing any Kubernetes file

This is genuinely useful advice and worth stealing:

PICTURE THIS

"Whenever you're writing a file, you need to visualise what's inside the thing you're creating."



```
Writing a POD?      → what's inside a pod? → CONTAINERS
                      so: containers → name, image, port, resources

Writing a SERVICE? → what does a service have? → a PORT, an IP, a URL
                      so: type + ports

Writing a REPLICASET? → what does it need? → HOW MANY + WHICH POD
                      so: replicas + selector + template
```

 *"That's how you need to think. You need to visualise for each one."*

The line that changes per service type

WATCH OUT

"This line will change with each service type."



```
type: NodePort      → + nodePort: 30080
type: ClusterIP     → nothing extra ("it's local, default, internal")
type: LoadBalancer → + the load balancer URL
```

The five types, previewed

NOTE

She listed all five and deferred the detail to Class 13:

#	Type	Note
1	ClusterIP	the default — <i>"just like we have a Docker default bridge network"</i>
2	NodePort	what today uses
3	LoadBalancer	
4	Headless	
5	ExternalName / DNS	

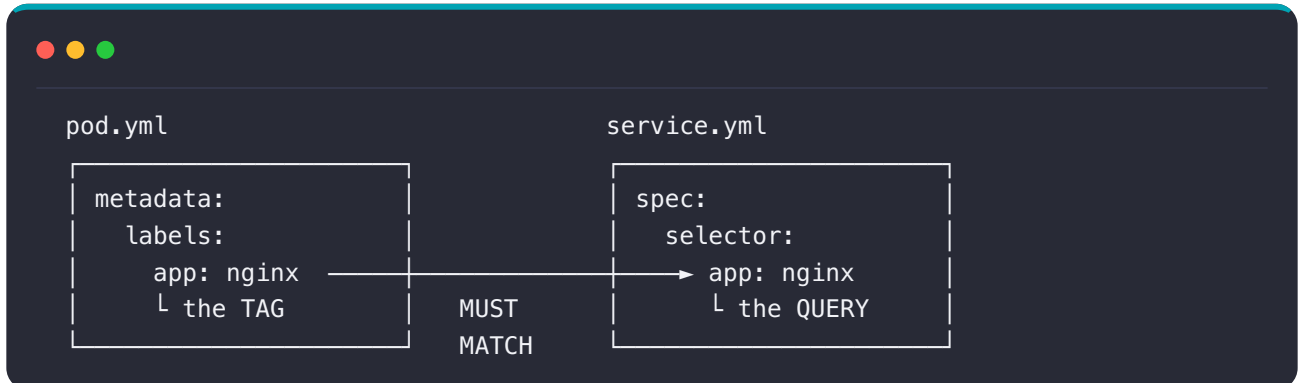
 **Her Docker analogy is a good one:** *"Just like we have the concept of network drivers — we have a default network, then we created a custom network. Same way, ClusterIP is the default. But we can't create a custom service type — there are only five predefined types."*



07 📄 Labels & Selectors — The Classic Interview Question

🎯 INTERVIEW ALERT

"There is an interview question here. What is the difference between label and selector, why do you need to use it, and which file do you use it in? It's a classic question."



	Goes in	Purpose
Labels	pod.yml (and pod templates)	"we are giving a tag to that pod"
selector	service.yml , ReplicaSet, Deployment	"to select specific services"

✅ LOCK THIS IN

Her instruction, verbatim, and it works: "Whatever I see here, blindly copy it and go inside your service. Inside a service you add a `selector` and paste it. Now both are connected."

🔥 Why this matters at real scale

She gave the most concrete picture of an enterprise Kubernetes estate in the whole course:

🔥 HARD MODE

"There's a long codebase I'm actually working on — a Helm chart. Inside the Helm chart you'll see specific folders for dev, stage, UAT, SIT, production, pre-release. And inside each one you'll see 26 environments, maybe 30 environments. In each environment you'll see 80+ services."

```
6 stages × ~28 environments × 80+ services ≈ 1-2 LAKH services
```

🔧 "And from those services you need to fetch one service, update a specific file, and do a rollout with zero downtime. At that time these tags and labels will help you."

 GOOD TO KNOW

That's why `kubectl get pods -l app=nginx` exists, and why nobody ever types an IP address.



08



Lab 1 — Pod + Service + Port Forwarding

DO THIS YOURSELF

The main hands-on of the class. She wanted to see the nginx page on every laptop.

Step 1 — the pod:

```
# pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx          # ← the TAG
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
      - containerPort: 80
```

Step 2 — the service:

```
# service.yml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx          # ← the QUERY, matching the tag above
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

Step 3 — apply both and look:

```
kubectl apply -f pod.yml
kubectl apply -f service.yml
kubectl get svc
kubectl get all
```

Step 4 — port-forward and open it:

```
kubectl port-forward svc/nginx-service 8080:80
# then open http://localhost:8080  ✓ nginx welcome page
```

💡 Both forms work: `svc/nginx-service` or `service/nginx-service`.

🎬 What the traffic actually does

🎬 WATCH IT HAPPEN

Her explanation of the chain, which is the point of the whole lab:

```
FRAME 1  your browser → localhost:8080
FRAME 2  kubectl port-forward tunnels it into the cluster
FRAME 3  → the SERVICE, on its port (80)
FRAME 4  → the service's selector finds pods labelled app=nginx
FRAME 5  → the POD, on its targetPort (80)
FRAME 6  → the CONTAINER, on its containerPort (80)
FRAME 7  nginx answers. Page renders. 🚀
```

💡 "How the port forwarding is working — that is the main concept that you need to understand. From the service, I'm exporting my node port; from this node I'm redirecting to the service's port number; and from there I port-forward directly to 8080."

💡 `-o wide` — the flag to use everywhere

```
kubectl get pods -o wide    # + IP, node, nominated node
kubectl get svc  -o wide    # + selector
kubectl get rs   -o wide
```

✓ LOCK THIS IN

"There is only one command, `-o wide`. At the end of any `get` command, just try to use it. You're able to see the pod's IP address, and which worker node it's scheduled on."





```
kubectl scale rs/<name> --replicas=5
kubectl scale deploy/<name> --replicas=5

# both naming styles work:
kubectl scale replicaset <name> --replicas=5
kubectl scale rs/<name> --replicas=5
```

INTERVIEW ALERT

Her question: "Why can't you scale a pod?"

 "Common sense use karo. If the pod has only one container, how can you scale one container? If there is only one thing inside a big circle, you can't scale it."

✅ **The real reason:** a Pod is a single instance. "Scaling" means *making more instances*, and only something that owns a **template** — a ReplicaSet or a Deployment — knows how to stamp out another one. A Pod has no template; it *is* the thing.



```
kubectl scale pod/<name> --replicas=3    ❌ no such thing
kubectl scale rs/<name> --replicas=3     ✅
kubectl scale deploy/<name> --replicas=3 ✅
```

CLASS MOMENT

And a genuinely good invitation to experiment: "Can anyone try something like `--replicas=-1` ? Have you tried? I used to do the same thing when I was learning. It's curiosity. Can't kill the curiosity." (Kubernetes rejects it — negative replicas are invalid.)



GOOD TO KNOW

On scale-down timing: "Sometimes it depends on the container. If the service is big or heavy, it will take some time to scale down." Pods get a grace period to shut down cleanly.





```
apiVersion: apps/v1      # ⚠ NOT v1 — anything that manages pods is apps/v1
kind: ReplicaSet

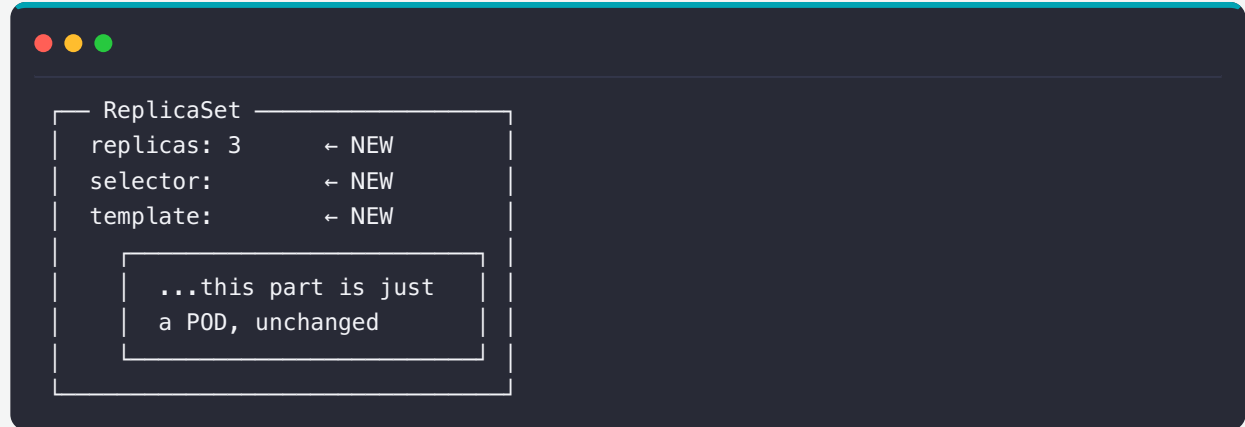
metadata:
  name: yatri-backend-rs  # convention: <name>-rs
  labels:
    app: yatri-backend

spec:
  replicas: 3              # ← NEW: how many
  selector:                # ← NEW: which pods are mine
    matchLabels:
      app: yatri-backend
  template:                # ← NEW: how to build one
    metadata:
      labels:
        app: yatri-backend
    spec:
      containers:
        - name: backend
          image: python:3.11-alpine
          command: ["python3", "-c", "import http.server; http.server.test(HandlerClass=http.server.SimpleHTTPRequestHandler, port=5000)"]
          ports:
            - containerPort: 5000
```

💎 The insight that makes ReplicaSet click

🗣️ SAID IN CLASS

"If I remove the `spec`, `replicas`, `selector` and `template` — then it equals `pod.yaml`. Correct?"



✅ "By just updating a `pod.yaml` like this, and changing the `kind`, you are going to create a `ReplicaSet`." Once you see it that way you never have to memorise the shape again.

⚠️ WATCH OUT

Note the `apiVersion` change: `v1` for a Pod, `apps/v1` for a ReplicaSet. "It shouldn't be `v1`, it should be `apps/v1`."

😊 CLASS MOMENT

On her one-line Python server: "I did not create an `app.py`. Whatever was inside `app.py`, I just pasted it over here. If you don't want to use this long command you can create your own `app.py` ... but I was so lazy, so I added it by one command." 😊

🏆 Lab 2 — ReplicaSet lifecycle

👤 DO THIS YOURSELF

Run the whole thing.

```
# clean up first
kubectl delete pod nginx-pod
kubectl delete svc nginx-service

# create
kubectl apply -f backend-rs.yml
kubectl get pods           # → ContainerCreating → 1 → 2 → 3 Running
kubectl get rs             # ★ now this returns something
kubectl get all

# scale it around
kubectl scale rs/yatri-backend-rs --replicas=5
kubectl get pods           # → 5
kubectl scale rs/yatri-backend-rs --replicas=0
kubectl get pods           # → scaling down...
kubectl scale rs/yatri-backend-rs --replicas=2

# delete
kubectl delete rs yatri-backend-rs
kubectl get pods           # → Terminating, then gone
```

🗣️ "Can you just play around with this command — `--replicas 0, 1, 2, 3`? You'll get to know how to scale replicas, how to scale your containers."

💡 GOOD TO KNOW

And a reality check: "Right now if you're working as a DevOps engineer, you will see thousands of ReplicaSets."

📌 NOTE

One gap she pointed out herself: a ReplicaSet alone gives you no external access. "If I do `-o wide` I can see the IP address, but I don't see any port forwarding. That means if I go to `:5000`, I'm not able to get localhost. For that I need to create a service — and add the same label."



11 🚀 Deployment & The Four Strategies

🎧 INTERVIEW ALERT

She showed the deployment file and then asked: "Now you tell me — why do we need `deployment.yaml` ?"

Why ReplicaSet isn't enough

🗣️ SAID IN CLASS

"By ReplicaSet you can't do migration. I mean, from a ReplicaSet how can you do a migration? You have replicas, you're putting a command in 1000 microservices — how are you going to do that?"

✅ **What ReplicaSet is limited to:** "ReplicaSet we only use for scale up, scale down, create and delete. But we need some mechanism that is actually managing those replicas at once — and that's why Deployment is there."

🧠 PICTURE THIS

The version-migration story: "Any frontend application — v1 is there and you need to update it to v2. Everyone who's worked on a live project related to migration knows: whenever we migrate, we migrate all the tags from v1 to v2, v2 to v3, v3 to v4. That's how we know this is the new version we released."

🌐 The Hong Kong → Singapore migration

She framed all four strategies around one scenario, and it makes them concrete:

🎧 INTERVIEW ALERT

"There is an infrastructure running on Hong Kong servers. Now you want to migrate to Singapore servers. You need to move all the workloads with 0% downtime. How are you going to do it?"

1 Rolling Update — "the basic one"

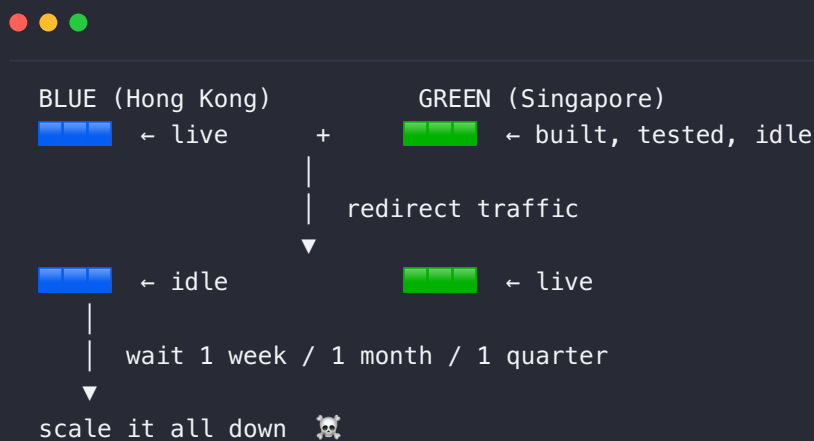


cost: 💰 downtime: none

🗣️ SAID IN CLASS

"You update it in one specific infrastructure, you bring the old pods down, create new pods, and then redirect the traffic."

2 Blue-Green — "zero downtime, double the bill"

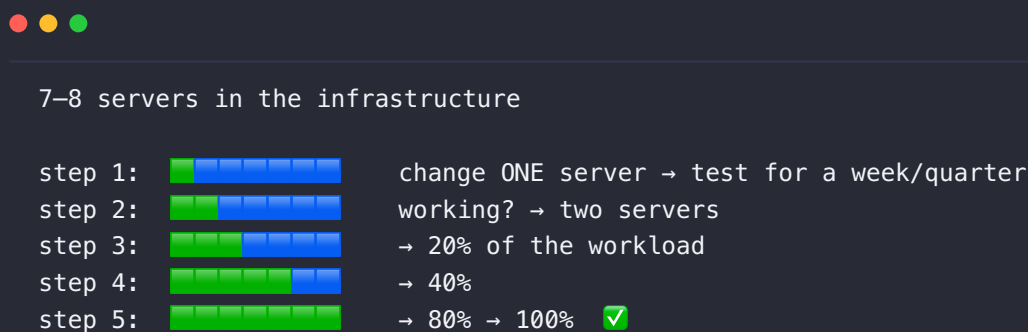


✓ LOCK THIS IN

"You create the same copy of the infrastructure, make it up and running, and the last task is to redirect your traffic from the existing one to the new one. And once the new one is working fine — after one month, maybe two months, maybe one week, maybe one quarter — then you scale down all the old replicas."

💰 **Her cost point, which is the real trade-off:** "One infrastructure is costing maybe 1 billion. And the same infrastructure again for 1 billion. So right now you're paying for 2 billion. That is the costliest part of blue-green. Pros: zero downtime. Cons: you're costing with your money."

3 Canary — "my favourite one" 🐣



✓ LOCK THIS IN

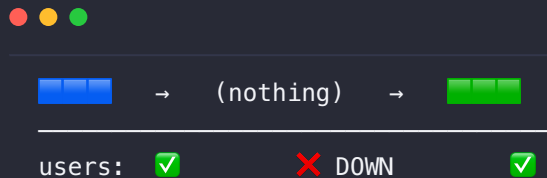
"You take one part of that infrastructure — let's suppose there are seven to eight servers. You put one server, make your changes, and test it. If it's working fine in one week, two weeks, a quarter — then you scale from one server to two servers. Again testing. Then three, four, five... 50%, 70%."

💰 **And the cost answer she made the class work out:** "What do you think regarding canary? There is no money lost — because we are not creating any other infrastructure. We're just testing on one particular component."

4 Recreate — "fourth one is there, but we are not using it"

NOTE

She named it and moved on: "Fourth one is there, but we are not using it — I'll tell you next lecture."
Here it is anyway, so the set is complete.



```
spec:
  strategy:
    type: Recreate      # no rollingUpdate block – nothing to tune
```

LOCK THIS IN

Kill everything, then start everything. It's the only strategy with real, visible downtime.

So why would anyone choose it? Because sometimes v1 and v2 genuinely cannot coexist:

Situation	Why coexisting breaks
A database schema migration	v1 expects the old columns, v2 expects the new ones. Both running = corrupted writes.
An app that takes an exclusive file lock	the second instance can't even start
A licence limited to one instance	the second one refuses to run

🧠 **In the Hong Kong → Singapore framing:** this is "shut down Hong Kong on Friday night, bring up Singapore on Saturday morning, accept the outage." Cheapest of the four, and the only one you'd have to announce to users.

Which one to pick

Strategy	Downtime	Cost	Rollback	Both versions live at once?
RollingUpdate	none	💰 +1 pod	gradual	⚠️ yes
Blue-Green	none	💰💰 double	★ instant	✗ no
Canary	none	💰 +1 pod	delete the canary	⚠️ yes, deliberately

Strategy	Downtime	Cost	Rollback	Both versions live at once?
Recreate	⚠️ yes	💰 none	redploy the old	❌ no

🎯 Which one to pick

Strategy	Downtime	Cost	Her verdict
RollingUpdate	none	💰	"the easy one"
Blue-Green	none	💰💰 double	"the costliest part"
Canary	none	💰	★ "my favourite" — no money lost
Recreate	⚠️ yes	💰	Class 12

✅ LOCK THIS IN

And who decides: "It depends which type of infrastructure you have and which client you're working for. One Kubernetes administrator will be there and he's deciding which strategy we need for migration, depending on client need."

🎯 INTERVIEW ALERT

Interview alert: "If you have ever given any DevOps interview or cloud interview, then this question will surely be there: can you explain the deployment types?"



12 🎬 maxSurge & maxUnavailable — The Long Version

This is where the two numbers were **first** taught, and she goes deeper here than in Class 12.

```
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

maxSurge — the ceiling

✓ LOCK THIS IN

"maxSurge means how many maximum replicas can be there during migration. If I add 1, then $3 + 1 = 4$. Maximum four replicas can be there."

🎬 WATCH IT HAPPEN

Her full walkthrough — 3 replicas, maxSurge 1:



```
START      [blue][blue][blue]      3 old

"3 replicas are running. Now I create a new replica with the new version."
STEP 1     [blue][blue][blue][green] 4 = the ceiling (3+1)

"I cannot create a fifth — my maxSurge is one. So from the three old
replicas, I terminate one, and then create another with the new version."
STEP 2     [blue][blue][green]      3
STEP 3     [blue][blue][green][green] 4 = ceiling again

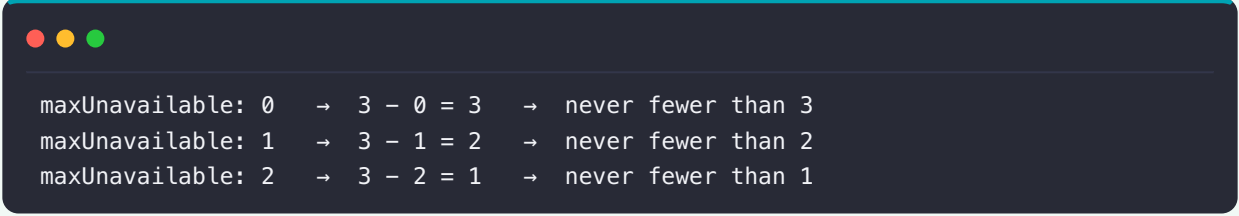
STEP 4     [blue][green][green]      3
STEP 5     [blue][green][green][green] 4

STEP 6     [green][green][green]      ✓ "now we have three new replicas.
                                         Our migration is done."
```

maxUnavailable — the floor

✓ LOCK THIS IN

"Maximum how many replicas can be unavailable. I set it to zero. That means $3 - 0 = 3$, so minimum three replicas should be up and running, anyhow."



```
maxUnavailable: 0 → 3 - 0 = 3 → never fewer than 3
maxUnavailable: 1 → 3 - 1 = 2 → never fewer than 2
maxUnavailable: 2 → 3 - 2 = 1 → never fewer than 1
```

🗣️ "I know this concept is a little bit tricky to understand. You need to write it down somewhere and then every day you need to recall it."

🎯 The question a student asked: why not set it unlimited?

🎯 INTERVIEW ALERT

"Why are we putting the tag? Why can't it be unlimited?"

✅ **Her answer, scaled up:** "I've only said three replicas — this is a small deployment for one service. But think there are a million services. In each, two services. So how many pods will be created? A million $\times$ 2. If you keep it unlimited, you will put the whole world's infrastructure into it."

🧠 **The real point:** these two numbers are a **budget**. Without a ceiling, a rolling update across a large estate would try to double your entire cluster simultaneously, and the cluster would run out of nodes.



13 ⚙️ Resources — requests vs limits

```
resources:
  requests:           # the bare minimum I'm GUARANTEED
    cpu: "100m"
    memory: "128Mi"
  limits:             # the ceiling I can NEVER exceed
    cpu: "250m"
    memory: "256Mi"
```

Means

requests	"bare minimum — in my container I need 100 millicore CPU and 128Mi memory"
limits	"maximum CPU I can allocate to my service after running is 250 millicore"

⚠️ What happens when you blow through them

⚠️ WATCH OUT

Two different errors, two different behaviours:

Exceed	Error	What happens
memory limit	OOMKilled — "out of memory, your kernel kills it"	container is killed 🪦
CPU limit	CPU quota exceeded	container is throttled (slowed, not killed)

🗣️ "These are the standard errors, I'm telling you."

GB vs GiB

INTERVIEW ALERT

She quizzed the class: "Do you know the difference between GiB and GB?"

```
1 GB = 1000 MB    (decimal)
1 GiB = 1024 MiB  (binary) ← what Kubernetes uses
```

✅ "Actually we are not using GB here — we are using GiB. Not MB, we are using Mi."

Why limits exist — the legacy-app problem

HARD MODE

"Some legacy applications are there. They're using 1 GB memory, then 25 GB memory, and then something happens inside that service and they occupy the complete server memory. That means 500 to 600 services are running on that particular server, and that one service is occupying every CPU and every memory. So what about the 499 other services running on it? This should not happen. That's why we put a limit."

GOOD TO KNOW

Her practical note: "Even if you don't want to use `limits`, that's okay — you can just use `requests`. But for best practices — in real infrastructure we specify both."



14 The OpenShift Outage Story

The best war story of the class, and it teaches a real operational concept.

HARD MODE

What happened. "A few months back we had a holiday. In our OpenShift cluster — you know OpenShift? Red Hat Enterprise Linux has a premium paid cluster, OpenShift. We use that cluster in our company."

"In that, almost around 700 to 800 pods were there, and they just went down. I don't know what happened. Immediately — midnight, maybe. And in the morning we got to know these things were happening. All the DevOps people were like, oh my God. Each person working on each environment, bringing up all the services."

🎯 The counterintuitive bit

🎯 INTERVIEW ALERT

"Scale up will not work at that time. Do you know why?"

✅ "Because if the scheduler has to schedule 700 pods, then the scheduler itself will go down. Because the scheduler is also one type of server, right? It's working on one server."

❌ WHAT DOESN'T WORK

```
scale all 800 at once
  ↓
scheduler is overwhelmed
  ↓
scheduler goes down 💀
now nothing can be scheduled
```

✅ WHAT THEY DID

```
BATCH PROCESSING
deploy 5 → 10 → 20 → 40 → 50
  ↓
one environment comes up
  ↓
move to the next
```

The recovery order

✅ LOCK THIS IN

Priority matters when you're recovering:

- ```
1. UAT and SIT ← "those are CRITICAL environments. Make them up first."
2. dev ← "usually for testing purposes"
```

🗣️ "So we take a batch of services. Batch processing — have you heard this concept in computer science? In batch processing we deploy five services, ten, twenty, forty, fifty — and then one environment is up. So developers can work on that one environment."

### 🧠 PICTURE THIS

**The lesson generalises:** the control plane has finite capacity too. In a mass outage, throttling your own recovery is faster than trying to do it all at once.

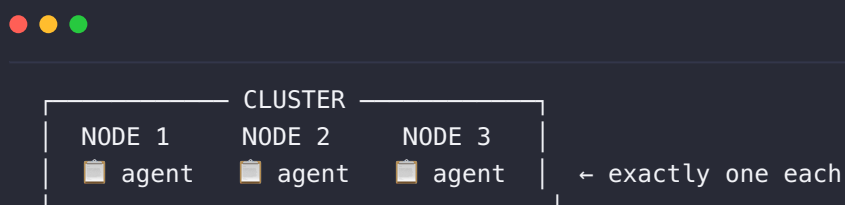


## 15 👤 DaemonSet — One Pod Per Node

```
apiVersion: apps/v1
kind: DaemonSet # ← the only real change from a Deployment
metadata:
 name: log-agent
spec:
 selector:
 matchLabels:
 app: log-agent
 template:
 metadata:
 labels:
 app: log-agent
 spec:
 containers:
 - name: agent
 image: busybox
```

### ✅ LOCK THIS IN

"Just one thing is changing here — `kind` is `DaemonSet`. That's it." (No `replicas` field — the node count is the replica count.)



- + new server added → 📁 agent appears automatically
- = server deleted → its agent's data is collected, then removed

### 🗣️ SAID IN CLASS

"If you're creating a server, once the server is created, automatically the `DaemonSet` will create one pod inside that server. If you're deleting a server, automatically the `DaemonSet` will take that container's data and then delete it."

**What you'd run in one:** "Maybe it's a sidecar container, maybe an agent, maybe a Falco container to check the security, maybe a log collector." She also named **FluentD** and **FluxDB** as security/logging agents.

### ⚠️ WATCH OUT

**Why she still couldn't demo it:** *"To do this practical we need cloud — we need two or three servers. Right now we have only one node. If I run it, it will work — but how can you check that it's working across two or three servers?"*

📌 **Her promise:** *"I'll give you a lab session, or maybe next session I'll create my own EC2 instances — in one cluster I'll create three servers, and then run this file."*



## 16 📝 Homework

### # Task

- 1 🚀 Run `deployment-v2.yaml` — the second version, to see a migration
- 2 🐛 **The pod lifecycle files.** She built one file per state: *"container creating, running, pending.yaml, completed, successful, crash loop, image pull error."* Run each, then `kubectl get pods` and watch the state
- 3 🐛 **The troubleshooting exercise** ★ — *"I added a misconfiguration in that file. Once you deploy it you'll see an error inside the container. Do the `exec` command, check, and resolve that error. You need to update something inside your file — maybe one word."* Two files
- 4 🏆 Redo **Labs 1 and 2** above
- 5 📖 Explore the new files she pushed to the repo

### 🗣️ SAID IN CLASS

**Why she built the lifecycle files:** *"Some people tell me 'okay ma'am, explain this' — these many stages are there, but no hands-on for the stages. So I created these files."*

### ⚠️ WATCH OUT

**Her warning about the pace:** *"Kubernetes is not an easy concept. After this session you need to recap the session plus you need to learn on your own. If you are not learning, then I'm telling you — it will be very hard for you."*

😬 *"Kubernetes is hard if you are not practising."*





## Commands

| Command                                                                | Does                                       |
|------------------------------------------------------------------------|--------------------------------------------|
| <code>kubectl get pods\ svc\ rs\ deploy\ nodes\ namespaces\ all</code> | list things                                |
| <code>kubectl get &lt;anything&gt; -o wide</code>                      | ★ + IP, node, selector                     |
| <code>kubectl get events</code>                                        | cluster-level log                          |
| <code>kubectl describe pod\ deploy\ node &lt;name&gt;</code>           | the <code>docker inspect</code> equivalent |
| <code>kubectl logs &lt;pod&gt;</code>                                  | container output                           |
| <code>kubectl exec -it &lt;pod&gt; -- sh</code>                        | shell inside                               |
| <code>kubectl apply -f &lt;file&gt;</code>                             | create <b>or</b> update                    |
| <code>kubectl create -f &lt;file&gt;</code>                            | create only — errors if it exists          |
| <code>kubectl delete pod\ svc\ rs\ deploy &lt;name&gt;</code>          | remove                                     |
| <code>kubectl scale rs/&lt;name&gt; --replicas=5</code>                | scale a ReplicaSet                         |
| <code>kubectl scale deploy/&lt;name&gt; --replicas=5</code>            | scale a Deployment                         |
| <code>kubectl port-forward svc/&lt;name&gt; 8080:80</code>             | reach a service from your browser          |

## The four ports

| Port                       | Belongs to    | File                      |
|----------------------------|---------------|---------------------------|
| <code>containerPort</code> | container     | pod / deployment template |
| <code>targetPort</code>    | pod           | service                   |
| <code>port</code>          | service       | service                   |
| <code>nodePort</code>      | node (server) | service ( 30000–32767 )   |

## apiVersion

| Kind                                           | apiVersion           |
|------------------------------------------------|----------------------|
| Pod, Service                                   | <code>v1</code>      |
| ReplicaSet, Deployment, DaemonSet, StatefulSet | <code>apps/v1</code> |

## Concepts

| Term                  | Means                                                                     |
|-----------------------|---------------------------------------------------------------------------|
| Node                  | a <b>server</b> added to a cluster                                        |
| Label                 | a tag <b>on a pod</b>                                                     |
| Selector              | a query <b>in a Service / ReplicaSet / Deployment</b>                     |
| Ingress               | one URL → thousands of services; exists to <b>save load-balancer cost</b> |
| Path-based routing    | <code>myapp.com/payments</code> → the payment service                     |
| ReplicaSet            | pod template + <b>replicas</b> + <b>selector</b> + <b>template</b>        |
| Deployment            | manages ReplicaSets + <b>migration strategies</b>                         |
| RollingUpdate         | gradual replacement — cheap, no downtime                                  |
| Blue-Green            | full duplicate infra — no downtime, <b>2x cost</b>                        |
| Canary                | 1 server → 20% → 40% → 80% → 100% — <b>no extra cost</b>                  |
| <b>maxSurge</b>       | ceiling: <code>replicas + maxSurge</code>                                 |
| <b>maxUnavailable</b> | floor: <code>replicas - maxUnavailable</code>                             |
| <b>requests</b>       | guaranteed minimum resources                                              |
| <b>limits</b>         | hard ceiling — exceed memory → <b>OOMKilled</b> ; CPU → <b>throttled</b>  |
| <b>Mi / Gi</b>        | binary units (1024), not 1000                                             |
| DaemonSet             | exactly one pod per node, automatically                                   |
| Batch processing      | recover a mass outage in waves, so the scheduler survives                 |



18



## Notes on the Transcript Itself

- **Instructor name:** the header credits "*Ritesh Prajapati*" — the **Scaler++ Chrome extension's developer**, not the teacher. The instructor is **Nensi Ravaliya** ([@Nency-Ravaliya](#)).
- 📌 **This replaces a wrong file.** Until now this folder contained a **byte-identical copy of Class 10's transcript**. These notes are written from the genuine Class 11 recording (8 September, *Kubernetes Networking & Services*).
- 📌 **She teaches several of these topics twice**, and both sets of notes cover them **in full** rather than pointing at each other — **maxSurge / maxUnavailable**, the **deployment strategies**, and **ReplicaSet vs Deployment** all appear here *and* in Class 12. The explanations differ because hers did: this class uses the **Hong Kong → Singapore migration** framing and the **cost comparison**; Class 12 re-

derives the same two numbers with a different worked example and adds the **rollout commands**.  
Read whichever lands better — or both, which is exactly how she taught it.

- 📌 **Ingress is introduced here** (§4), conceptually and as a cost argument. That's the topic Class 12's folder title promised but never delivered. The hands-on still hasn't happened.
- ⚠️ **Roughly the middle third is lab time and largely unusable** — she walks the room while students run commands, and the transcript becomes *"Thank you"*, Portuguese, Spanish and fragmentary English. The troubleshooting exercise and some file names are only partially recoverable; that's flagged rather than invented.
- 📌 **A stretch of the maxSurge explanation is in Hindi** (she re-explains it for the room). Its content matches the English walkthrough immediately before it, so it's folded into §12 rather than listed separately.
- Garbled terms corrected: "Qx TI / Qtl / cube ctl / kipsutl / C++" → **kubectl**, "Scc / svc" → **svc**, "megsearch / maxsourds / max search / next search" → **maxSurge**, "mechs unavailable" → **maxUnavailable**, "blue deployment / BlueBrain / green blue" → blue-green, "dml set / demon set" → **DaemonSet**, "Farco" → **Falco**, "FluxDB" → likely **FluentD/Fluent Bit**, "notebook / node port" → **NodePort**, "OOO / OO" → **OOMKilled**, "billy core" → **millicore**, "Yathri / yacht 3" → **yatri-backend**, "1 NAR" → **one hour**, "open shift" → **OpenShift**.